ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

École Marocaine des Sciences de l'Ingénieur (EMSI) – Rabat
Filière Intelligence Artificielle & Data Science (5IASD)

Compte rendu de TP05

AutoML : création et déploiement
d'un modèle pour la détection de fraude

Module : DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES

Réalisé par : **Mohamed BENHEDDANE**
Yassir DRISSI
Zineb BAQLOUL
Filière : 5IASD
Groupe : G8
Encadrant : Pr. H. Naciri

2 novembre 2026

Table des matières

1 Introduction et protocole expérimental **2**

2 Réalisation et résultats **3**
2.1 Recherche AutoML et extraction du gagnant 3
2.2 Packaging et enregistrement dans le registre 4
2.3 Promotion et démarrage des deux services 5
2.4 Consommation HTTP et validation de la signature 6

3 Comparaison et décision de production **7**

4 Réponses aux questions d'évaluation **8**
Question 1 : le choix de production 8
Question 2 : le coût d'un point de terminaison cloud 8
Question 3 : latence locale et latence cloud 8
Question 4 : le moteur change de version 9
Question 5 : réversibilité 9
Question 6 : skops ou cloudpickle 9

5 Conclusion et références **10**

1 Introduction et protocole expérimental

Le TP05 prolonge la recherche AutoML du TP04 par une chaîne de mise en service complète : entraînement, packaging, enregistrement dans MLflow, promotion par alias, service HTTP et consommation par un client externe. Nous comparons le gagnant de FLAML à la forêt aléatoire manuelle du TP2 sur quatre axes : qualité, latence, taille et explicabilité.

Le modèle manuel obtient le meilleur F1 de test, soit 0,8669 contre 0,8561 pour AutoML. XGBoost, retenu par FLAML, est cependant 47,9 fois plus compact et présente une latence médiane de 30 à 31 ms, contre 92 à 118 ms pour le manuel. La décision de production doit donc intégrer ce compromis.

Données et comparaison équitable

Le script `donnees.py`, repris du TP04, génère 5 000 transactions synthétiques avec une graine fixée à 7 ; 568 transactions sont frauduleuses. Les variables sont `montant`, `anciennete`, `freq_24h` et `nuit` ; la cible est `fraude`.

Les deux scripts utilisent `train_test_split` avec `test_size=0.25` et `random_state=42`, sans stratification, comme le sujet. La partition comporte 3 750 observations d'entraînement et 1 250 de test, dont 134 fraudes. FLAML sélectionne ses réglages sur sa validation interne ; le test externe ne sert pas à régler les hyperparamètres.

Approche	**Durée machine**	**F1 test**	**Format MLflow**
Manuel : Random Forest 500/16	2,86 s	0,8669	skops
AutoML : XGBoost, budget 90 s	90,00 s	0,8561	cloudpickle

Table 1 – Résultats de la série d'exécutions du TP05.

Environnement et traçabilité

Les exécutions utilisent Windows, Python 3.14.7, FLAML 2.7.0, MLflow 3.17.2, scikit-learn 1.9.1, XGBoost 2.1.4, pandas 3.1.0 et NumPy 2.5.4. FLAML fonctionne directement : le plan B n'a pas été utilisé. La recherche est limitée à deux fils de calcul et à un budget de 90 secondes.

Le projet est enregistré dans `C:/Users/pc/Documents/mlops-workspace/tp5-automl-deploy`. Le registre du TP05 écoute sur le port **5005**, car le port 5000 est utilisé par le TP04. Les services de prédiction utilisent les ports **8100** et **8101**, conformément au sujet. Le parcours cloud facultatif n'a pas été exécuté.

2 Réalisation et résultats

2.1 Recherche AutoML et extraction du gagnant

Le script `src/automl_deploy.py` lance FLAML pendant 90 secondes avec `metric='f1'` et `seed=42`. Le gagnant est un `XGBClassifier` comportant 322 arbres et au plus quatre feuilles par arbre. Son F1 de test vaut 0,8561. L'estimateur extrait avec `automl.model.estimator` produit exactement les mêmes prédictions que l'objet FLAML sur les 1 250 lignes du test.

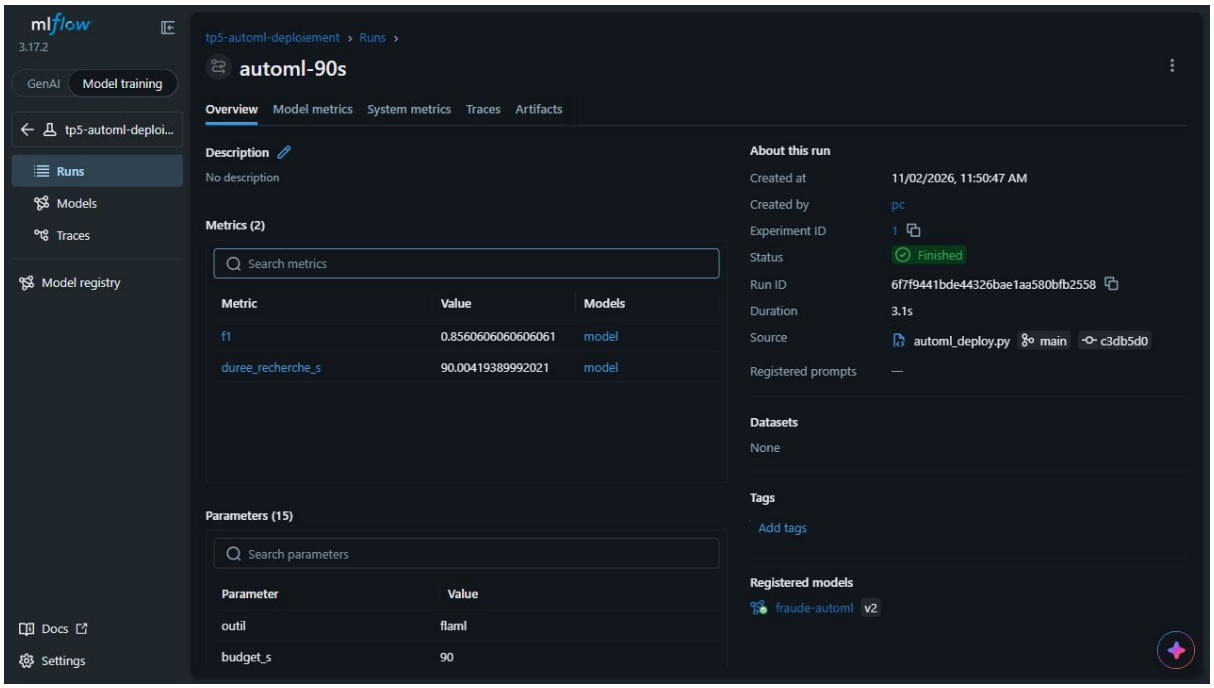


FIGURE 1 – Capture 1a : run automl-90s, métrique F1 et durée de recherche dans MLflow.
Fichier : `captures/capture1_run.jpg`

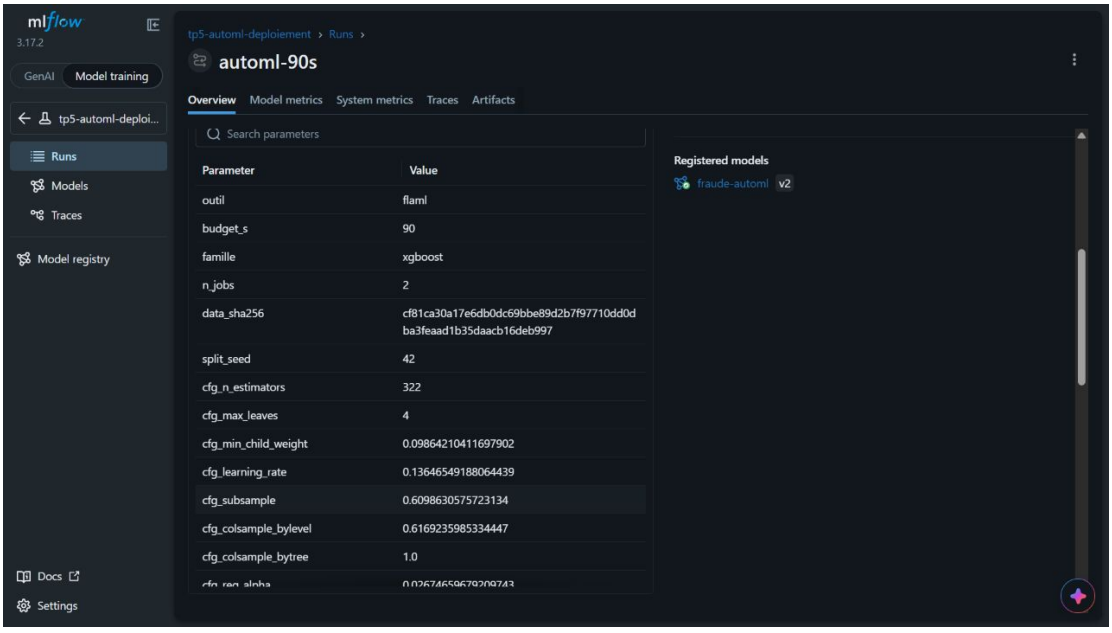


FIGURE 2 – Capture 1b : famille xgboost et hyperparamètres gagnants.
Fichier : `captures/capture1_parametres.jpg`

2.2 Packaging et enregistrement dans le registre

Le gagnant est enregistré sous `fraude-automl`, version 2. Une première version avait été enregistrée avant la correction de l'encodage des sorties Windows ; la série retenue dans ce rapport correspond à la seconde exécution. Le modèle manuel est enregistré sous `fraude-manuel`, version 1.

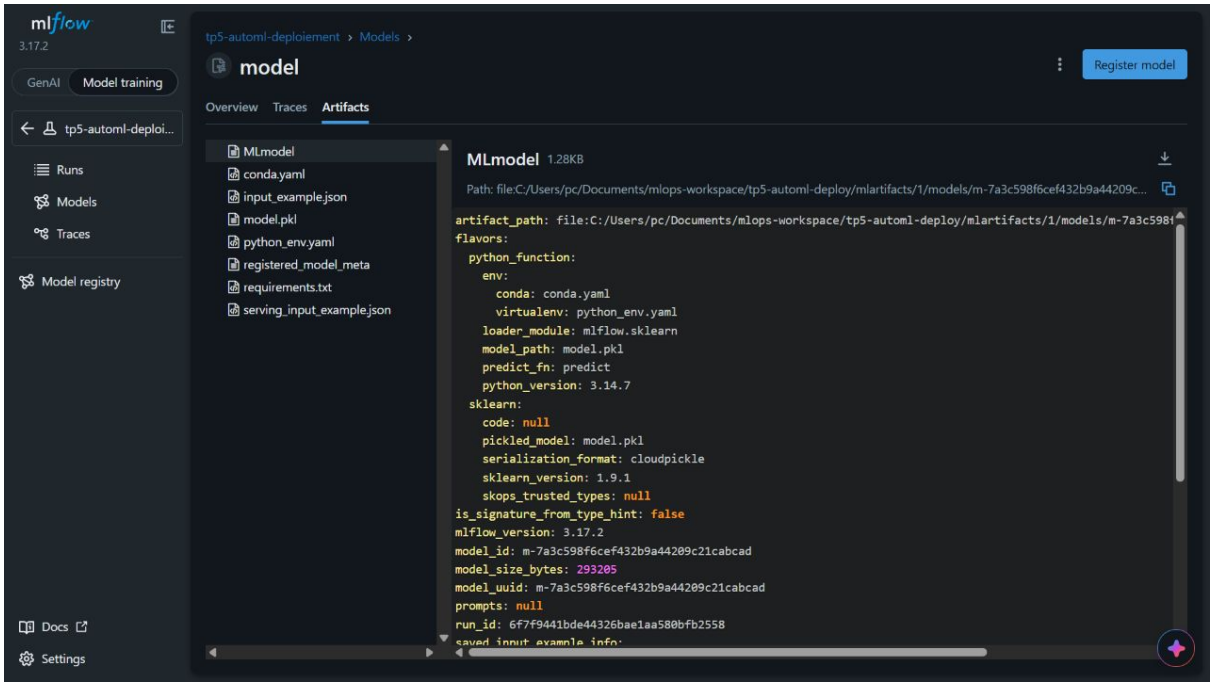


FIGURE 3 – Capture 1c : fichier MLmodel du gagnant, flavors et sérialisation cloudpickle.
Fichier : `captures/capture1_mlmodel.jpg`

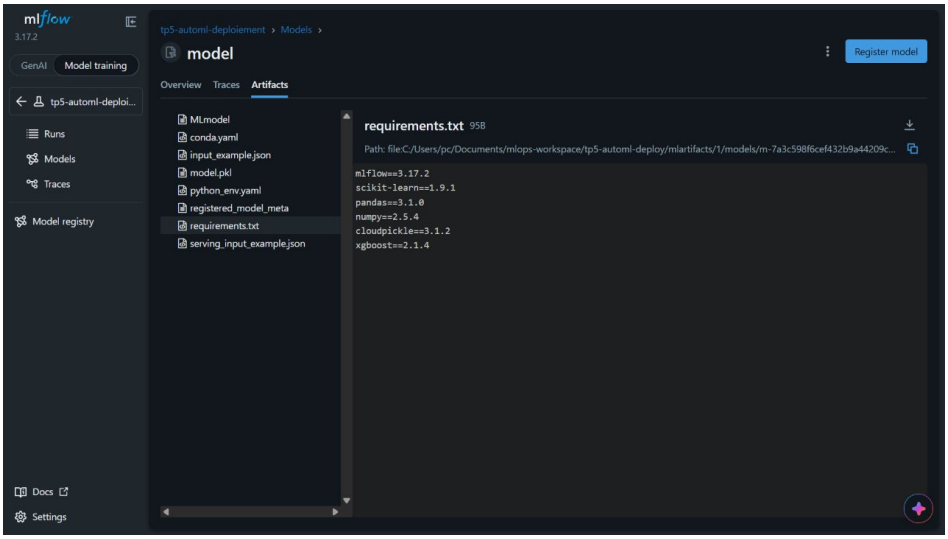


FIGURE 4 – Dépendances enregistrées : XGBoost 2.1.4 figure dans requirements.txt.
Fichier : `captures/capture1_dependances.jpg`

Les flavors `sklearn` et `python_function` permettent le chargement natif et le service HTTP. La signature impose `montant` en double et les trois autres variables en entiers longs. Le Random Forest utilise explicitement `serialization_format='skops'` et le type autorisé `sklearn.tree._tree.Tree`.

2.3 Promotion et démarrage des deux services

Le script `src/promouvoir.py` sélectionne la version la plus récente de chaque modèle, vérifie son état `READY`, puis lui attribue l'alias `production`. Les services utilisent cet alias et n'ont pas besoin d'un numéro de version codé en dur.

```
mlflow models serve -m "models:/fraude-automl@production" -h 127.0.0.1 -p 8100 --env-manager local
mlflow models serve -m "models:/fraude-manuel@production" -h 127.0.0.1 -p 8101 --env-manager local
```

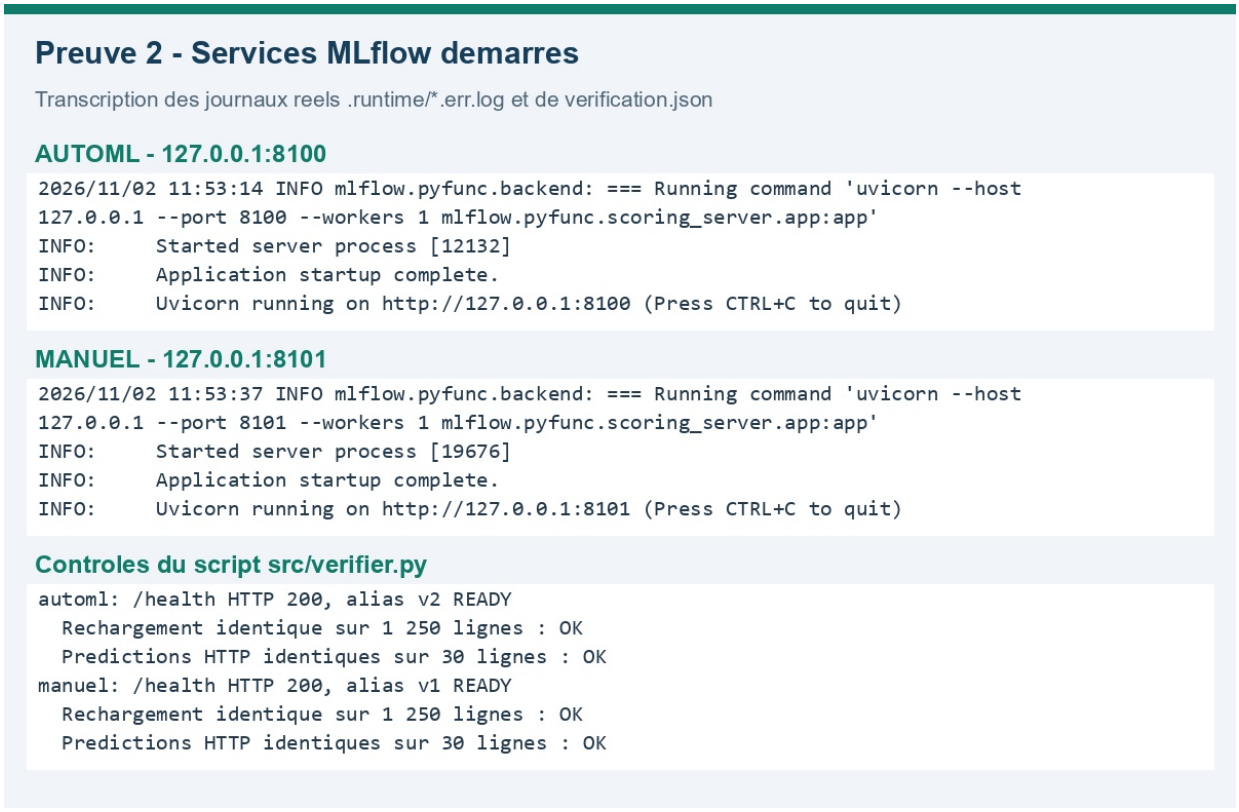


FIGURE 5 – Preuve 2 : démarrage des deux services et vérification des points de terminaison. Fichier : `captures/capture2_services.jpg`

Cette figure est une transcription des journaux réels d'exécution et des contrôles HTTP ; elle ne représente pas des fenêtres de terminaux. Les sorties originales sont conservées dans `.runtime/automl.err.log` et `.runtime/manuel.err.log`.

Le script `src/verifier.py` confirme les réponses HTTP 200 de `/health`. Le rechargement depuis les deux alias reproduit toutes les prédictions du test local. Pour chaque service, un lot de 30 transactions envoyé par HTTP donne également les mêmes prédictions que le modèle local.

Un alias est résolu au démarrage : modifier `production` ne remplace pas le modèle déjà chargé en mémoire. Après promotion ou retour à une ancienne version, les services doivent être redémarrés.

2.4 Consommation HTTP et validation de la signature

Le client envoie une transaction suspecte : `montant=8500.0`, `anciennete=3`, `freq_24h=9`, `nuit=1`. Les deux services répondent HTTP 200 et `predictions=[1]`.

Chaque série comprend dix appels d'échauffement non mesurés, puis 200 appels chronométrés avec `perf_counter`. Le client vérifie les réponses ; deux séries sont exécutées par service, soit 800 appels mesurés au total. Le p95 utilise `statistics.quantiles(n=20)`, comme dans le sujet.

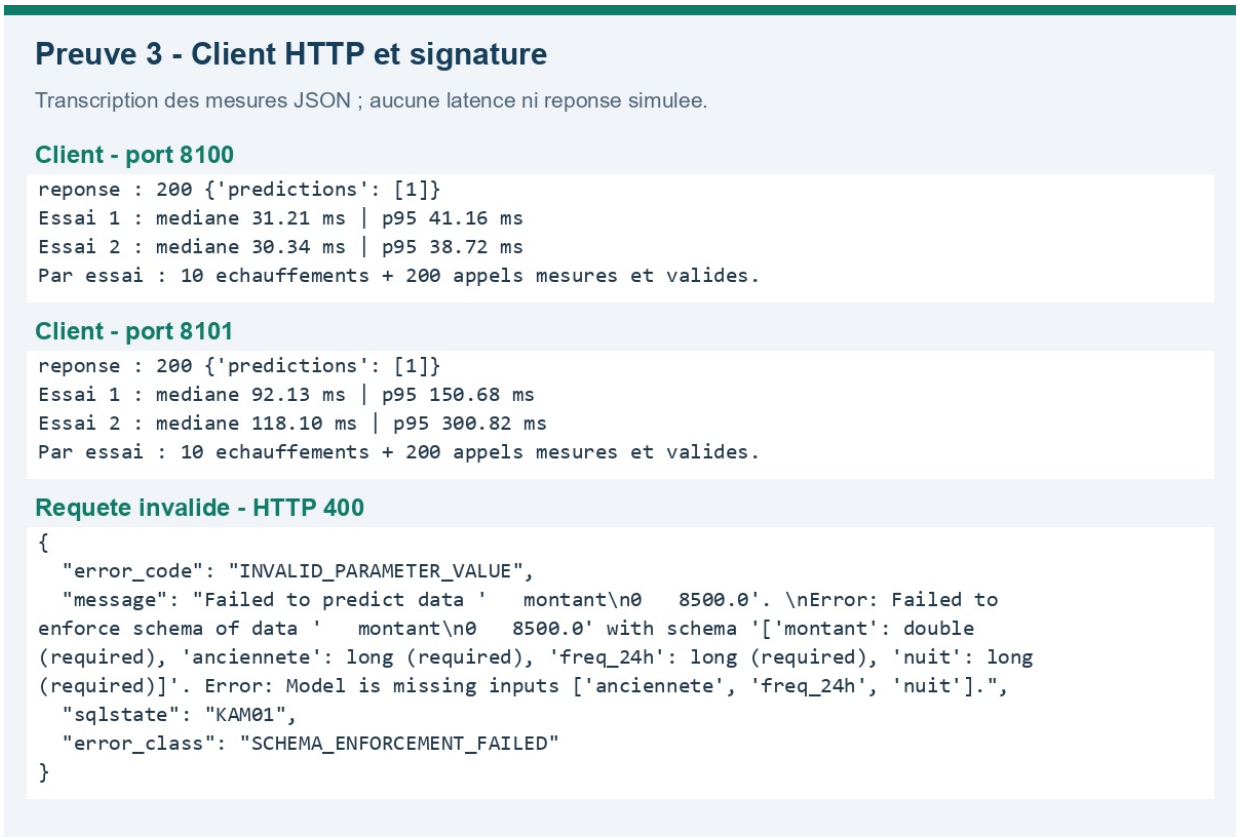


FIGURE 6 – Preuve 3 : sorties du client, deux répétitions par service et réponse à la requête invalide. Fichier : `captures/capture3_client.jpg`

Transcription des résultats JSON mesurés et de la réponse HTTP réelle ; aucune latence n'est simulée.

La requête contenant seulement `montant` est rejetée avec le statut **400**. Le message indique les colonnes manquantes : `anciennete`, `freq_24h` et `nuit`. La signature protège la forme de l'entrée ; elle ne garantit pas la validité métier de la transaction.

Les latences incluent la connexion locale, la sérialisation JSON, la validation du schéma et l'inférence. Les appels sont séquentiels, sans session persistante ni charge concurrente générée par le client. Le temps du modèle seul n'est donc pas isolé. La variabilité entre les deux séries du manuel rappelle la sensibilité des mesures à l'état du poste.

3 Comparaison et décision de production

Les valeurs ci-dessous proviennent de `reports/tp5_comparaison.md`. Les tailles sont mesurées sur les copies locales `joblib`, pour comparer le même format, et non sur l'archive skops du registre. Un Mio correspond à 1 048 576 octets.

Critère	**Manuel (RF 500/16)**	**AutoML (FLAML 90 s)**
Famille	Forêt aléatoire	XGBoost
F1 sur le test	0,8669	0,8561
Précision / rappel	0,8837 / 0,8507	0,8692 / 0,8433
Temps machine	2,86 s (entraînement final)	90,00 s (recherche et réentraînement)
Temps humain	Non mesuré au TP2 ; à compléter	Non mesuré ; préparation et analyse à inclure
Médiane, essai 1	92,13 ms	31,21 ms
Médiane, essai 2	118,10 ms	30,34 ms
p95, essai 1	150,68 ms	41,16 ms
p95, essai 2	300,82 ms	38,72 ms
Taille du fichier	13,374 Mio	0,279 Mio
Explicabilité	Importances ; 500 arbres, profondeur maximale 16	Importances ; 322 arbres, au plus 4 feuilles par arbre ; SHAP envisageable
Format du registre	skops	cloudpickle

TABLE 2 – Comparaison sur les mêmes données et les deux séries de mesures HTTP.

Décision en cinq lignes

Nous retenons AutoML pour son compromis entre qualité, latence et taille.
Le F1 vaut 0,8561 contre 0,8669 ; 21 fraudes sont manquées, contre 20.
Ses médianes de 30 à 31 ms et sa taille 47,9 fois plus faible sont des atouts.
Les importances sont accessibles ; le manuel est conservé pour un retour arrière.
Ce choix exige une validation métier des erreurs et un objectif de latence.

Lecture des erreurs

Le manuel produit 1 101 vrais négatifs, 15 faux positifs, 20 faux négatifs et 114 vrais positifs. AutoML produit respectivement 1 099, 17, 21 et 113. Sur ce test, l'écart correspond à une fraude supplémentaire manquée et deux fausses alertes supplémentaires pour AutoML. Il doit être confirmé sur d'autres partitions ou sur des données temporelles représentatives ; un seul découpage ne démontre pas une différence généralisable.

4 Réponses aux questions d'évaluation

Question 1 : le choix de production

Nous déploierions XGBoost issu de FLAML pour ce TP. Il est 47,9 fois plus compact, environ trois à quatre fois plus rapide en médiane dans nos séries, et son p95 reste inférieur à celui du manuel. Ces résultats compensent, dans notre choix pédagogique, une baisse de F1 de 0,0109. Le modèle manuel reste préférable si le coût d'une fraude manquée domine les contraintes de rapidité et de stockage.

Les autres critères sont la mémoire, le coût d'exploitation, le rappel des fraudes, la facilité de maintenance, la traçabilité et l'explicabilité. Les importances globales sont accessibles pour les deux ensembles d'arbres ; une analyse locale avec SHAP peut compléter l'étude. Une importance ne démontre pas une relation causale. Avant un usage réel, il faudrait fixer le coût des erreurs et valider la qualité sur une période indépendante.

Question 2 : le coût d'un point de terminaison cloud

La page officielle de tarification a été consultée le **2 novembre 2026** [1]. Hypothèse illustrative : une machine `n1-standard-2`, dans `us-central1`, à la demande, sans remise et avec un modèle déployé en continu. Le tarif de prédiction consulté vaut **0,1095 USD par heure**. Pour 30 jours :

$$C = 30 \times 24 \times 0{,}1095 = \boxed{78{,}84 \text{ USD}}.$$

Une base de 730 heures donne 79,94 USD. Une machine active reste facturée sans requêtes. Le calcul exclut entraînement, stockage, réseau, taxes et explications supplémentaires ; il ne constitue pas un devis pour `europe-west1`. La page indique que l'inférence AutoML tabulaire suit les tarifs de prédiction des modèles personnalisés. Aucune ressource cloud payante n'a été créée pour le TP.

Question 3 : latence locale et latence cloud

Les médianes AutoML locales valent 31,21 et 30,34 ms ; celles du manuel, 92,13 et 118,10 ms. Le cloud ajoute généralement le trajet réseau jusqu'à la région, TLS, l'authentification, le routage et des files d'attente éventuelles. Ces éléments augmentent le temps aller-retour, même si l'inférence elle-même reste rapide.

Une machine distante plus puissante peut réduire le calcul du modèle. La latence cloud n'est donc pas une constante que l'on peut ajouter aux mesures locales. Il faut comparer un client, une région, une charge et une taille de lot fixés. Aucun endpoint cloud n'ayant été testé ici, nous ne présentons pas de latence cloud mesurée.

Question 4 : le moteur change de version

Un nouvel entraînement peut explorer d'autres familles, retenir de nouveaux hyperparamètres et produire des scores différents, même avec les mêmes données. L'évolution du runtime peut aussi modifier le chargement ou le comportement d'un modèle servi. Le risque est une régression de qualité, une incompatibilité de dépendances ou une perte de reproductibilité.

Il faut conserver les versions du moteur et des bibliothèques, les données et leur empreinte, la configuration gagnante, la signature, le modèle et les résultats d'évaluation. La mise à jour doit être testée avant promotion. Le registre et les anciennes versions permettent de préparer un retour arrière ; il faut également conserver leur environnement d'exécution.

Question 5 : réversibilité

Il faut vérifier dès le choix de la plateforme les formats d'export et le droit d'exécuter le modèle hors du fournisseur. Lorsque l'export est possible, nous conserverions le modèle, le prétraitement, le schéma d'entrée, les dépendances et un jeu de référence, puis nous testerions l'inférence hors plateforme pour vérifier l'équivalence.

Un format interopérable tel qu'ONNX peut être envisagé si le modèle est compatible. Si l'export n'est pas possible, les données, les labels et un pipeline indépendant doivent permettre de réentraîner une solution de remplacement. Une simple API de prédiction ne garantit pas la réversibilité. Dans ce TP, les contrôles de rechargement depuis MLflow constituent une première vérification locale.

Question 6 : skops ou cloudpickle

Le gagnant de FLAML peut provenir de LightGBM, XGBoost ou scikit-learn. Dans ce TP, `cloudpickle` simplifie le packaging de l'estimateur via le flavor sklearn. La référence manuelle, connue à l'avance, est enregistrée explicitement en `skops`, avec le type `sklearn.tree._tree.Tree` autorisé. Le format et les dépendances sont conservés dans `MLmodel` et `requirements.txt` [2].

Les formats pickle et cloudpickle peuvent exécuter du code lors de la désérialisation. Leur chargement doit être limité à des artefacts de confiance. Cet usage reste acceptable ici dans un cadre local contrôlé : nous entraînons nous-mêmes le modèle et conservons son artefact dans notre registre. Skops permet de contrôler les types chargés ; élargir la liste de confiance sans examen réduit cette protection [3]. Aucun de ces formats ne dispense de conserver des versions compatibles des bibliothèques.

5 Conclusion et références

Le TP05 met en service deux modèles à travers le même registre et le même protocole HTTP. L'extraction du gagnant FLAML conserve exactement les prédictions de l'objet AutoML. Les alias `production`, les signatures et les vérifications automatisées assurent la cohérence entre entraînement, enregistrement et consommation.

La référence manuelle obtient le meilleur F1, mais XGBoost offre une taille plus faible et des latences nettement meilleures dans cette expérience. Nous retenons donc AutoML comme compromis pédagogique, en conservant le manuel comme solution de référence. La validation des coûts des faux négatifs, de la dérive et des objectifs de service resterait indispensable avant une production réelle.

Livrables et état du projet

Le dossier `src/` contient l'entraînement, la promotion, le client et les contrôles. Les fichiers `reports/*_resultats.json`, `latence_*.json` et `verification.json` conservent les résultats bruts. Le tableau est disponible dans `reports/tp5_comparaison.md`. Les métadonnées et les dépendances des modèles sont copiées dans `reports/artefacts/`.

Le dépôt Git local comprend les étapes du TP. Les données régénérables, les modèles binaires, le registre SQLite, les artefacts MLflow et l'environnement virtuel sont exclus du suivi. Les services locaux du TP05 ont été arrêtés après les mesures. Les temps humains ne sont pas inventés et restent à compléter par le groupe. La publication GitHub et l'envoi du rapport n'ont pas été effectués.

Sources documentaires

Les références suivantes ont été consultées le 2 novembre 2026. Le chiffrage cloud est une estimation sous hypothèses ; aucune performance cloud n'a été mesurée.

[1] Google Cloud : tarification Vertex AI / page actuelle de la plateforme ; rubriques *Tabular data* et *Prediction and explanation*, région `us-central1`.
https://cloud.google.com/products/gemini-enterprise-agent-platform/pricing

[2] MLflow : API `mlflow.sklearn`, formats de sérialisation et `skops_trusted_types`.
https://mlflow.org/docs/latest/api_reference/python_api/mlflow.sklearn.html

[3] scikit-learn : *Model persistence*, sécurité et limites de compatibilité.
https://scikit-learn.org/stable/model_persistence.html